Kiểm thử Bảo mật Phân quyền (RBAC/ABAC)

Bảo vệ Đồ án Tốt nghiệp - Dự án ScamphoneApp

🎙️ **Kịch bản nói:** *Chào quý thầy cô trong hội đồng, em là sinh viên thực hiện đồ án. Hôm nay, em xin phép trình bày báo cáo bảo vệ đồ án tốt nghiệp với chuyên đề: "Kiểm thử Bảo mật Phân quyền (RBAC/ABAC) áp dụng trên hệ thống Backend của ScamphoneApp".*

Nội dung Trình bày

- **Phần 1:** Tổng quan & Phương pháp tiếp cận
- **Phần 2:** Mô hình Đo lường học thuật
- **Phần 3:** Trọng tâm Kịch bản Kiểm thử
- **Phần 4:** Báo cáo Khiếm khuyết (Bug Reports)
- **Phần 5:** Báo cáo Tổng hợp & Kết luận

🎙️ **Kịch bản nói:** *Cấu trúc bài báo cáo của em gồm 5 phần chính: Đầu tiên là Phương pháp tiếp cận, tiếp đến là Công thức đo lường độ bao quát, Phần 3 em sẽ demo trực tiếp kịch bản kiểm thử, Phần 4 phân tích các Bug nghiêm trọng và cuối cùng là Tổng kết kết quả đạt được.*

Phương pháp Tiếp cận

- **Black-box Testing (Kiểm thử Hộp đen):** Đóng vai trò là người dùng hoặc Hacker, sử dụng công cụ Postman để tương tác độc lập với các API Endpoints. Khảo sát dữ liệu phản hồi (Response) từ Server.
- **White-box Testing (Kiểm thử Hộp trắng):** Can thiệp sâu vào cấu trúc mã nguồn Node.js. Viết kịch bản tự động bằng thư viện Jest nhằm soi chiếu logic Code và Middleware bảo vệ.
- **Mục tiêu:** Xác minh tính toàn vẹn của hệ thống từ lớp vỏ giao tiếp (API) đến tận lõi xử lý Logic (Source Code).

🎙️ **Kịch bản nói:** *Để đảm bảo hệ thống an toàn tuyệt đối từ ngoài vào trong, em đã áp dụng kết hợp cả hai phương pháp: Black-box testing thông qua Postman để mô phỏng Hacker khai thác qua API, và White-box testing sử dụng Jest để rà soát trực tiếp các lỗ hổng logic ẩn sâu bên trong mã nguồn Node.js.*

Kỹ thuật Kiểm thử Áp dụng

Đoán lỗi (Error Guessing)

Kỹ thuật khai thác các Payload dị dạng, cố tình loại bỏ các trường dữ liệu bắt buộc nhằm kiểm tra rào chắn Validation của hệ thống.

Phân vùng (Equivalence)

Phân vùng tương đương tập trung vào việc cách ly ranh giới quyền hạn. Tách biệt chặt chẽ các tác vụ thuộc nhóm Guest, User và Admin.

Bảo mật (Security)

Mô phỏng các cuộc tấn công Cyber Security thực tế như: Giả mạo chữ ký JWT Token, và khai thác lỗ hổng thao túng IDOR/BOLA.

🎙️ **Kịch bản nói:** *Ba kỹ thuật lõi xuyên suốt đồ án là: Đoán lỗi để bẻ gãy đầu vào, Phân vùng tương đương để xác định ranh giới giữa User thường và Admin, và đặc biệt là kỹ thuật Kiểm thử Bảo mật tập trung vào việc bẻ khóa JWT và khai thác lỗ hổng xem trộm dữ liệu IDOR.*

Mô hình Đo lường Học thuật

1. API Endpoint Coverage

Công thức đánh giá mức độ bao phủ toàn diện các điểm cuối giao tiếp của dịch vụ Backend.

$$C = \frac{API_{Tested}}{API_{Total}} \times 100\ \%$$

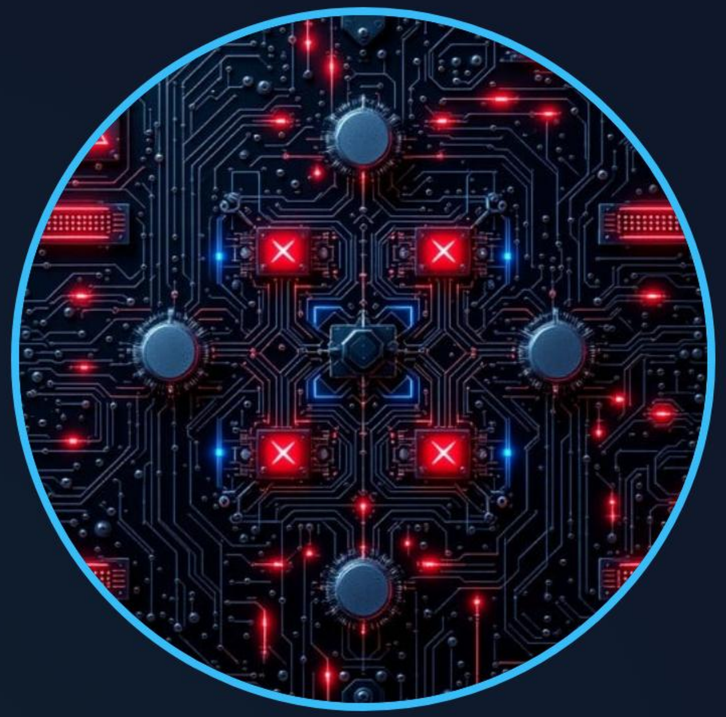
2. Role-Access Matrix Coverage

Đo lường tính chặt chẽ của ma trận phân quyền, đối chiếu mọi vai trò với tài nguyên hệ thống.

$$C = \frac{Pairs_{Tested}}{Pairs_{Total}} \times 100\ \%$$

🎙️ **Kịch bản nói:** *Để đảm bảo tính học thuật, em sử dụng hai công thức cốt lõi: API Endpoint Coverage nhằm đảm bảo không có API nào bị bỏ sót, và Role-Access Matrix Coverage nhằm định lượng số lượng ma trận quyền hạn đã được rà soát trên tổng số ma trận thiết kế.*

Áp dụng vào Scamphone



- **Bao phủ toàn diện 100%:** Các công thức trên được áp dụng trực tiếp để thiết kế ra bộ 43 kịch bản kiểm thử, quét sạch 100% Core API của Scamphone.
- **Xác thực Ma trận Quyền:** Quét toàn bộ các nhánh giao tiếp chéo giữa 3 nhóm đối tượng chính: Khách vãng lai (Guest), Khách hàng (User), và Quản trị viên (Admin).
- **Phát hiện Lỗ hổng:** Trọng tâm xoáy sâu vào lỗ hổng phân quyền (Broken Access Control) - Nút thắt nguy hiểm nhất trong kiến trúc Microservices.

🎙️ **Kịch bản nói:** *Dựa vào công thức đo lường, em đã xây dựng 43 kịch bản kiểm thử bao phủ toàn bộ 100% API của Scamphone. Hệ thống quét sạch ma trận quyền giữa Guest, User và Admin, đặc biệt tập trung tìm diệt lỗi Broken Access Control.*

Quy mô 43 Kịch bản Kiểm thử

Hệ thống chia làm 5 Module chính yếu, trong đó Module Xác thực (Auth) và Sản phẩm (Product) chiếm tỷ trọng Test Cases cao nhất nhằm siết chặt cổng an ninh đầu vào.

🎙️ **Kịch bản nói:** *Về quy mô, 43 Test Cases được chia đều cho 5 Module huyết mạch. Biểu đồ cho thấy Module Xác thực và Sản phẩm sở hữu số lượng kịch bản cao nhất, vì đây là "cửa ngõ" an ninh dễ bị Hacker dòm ngó đầu tiên.*

Demo Kịch bản: Happy Path

- **Mã Kịch Bản:** `TC-ORD-01` - Khởi tạo Đơn hàng Thành công.
- **Mục tiêu:** Đảm bảo khách hàng tạo giỏ hàng trơn tru khi thông tin hợp lệ.
- **Dữ liệu Đầu vào (Input):** Token ủy quyền hợp lệ của User + Payload Đơn hàng (danh sách sản phẩm, địa chỉ giao hàng đầy đủ).
- **Kết quả Mong đợi (Expected):** Hệ thống lưu Database và trả về mã HTTP `201 Created`.
- **Kết quả Thực tế (Actual):** Máy chủ ghi nhận dữ liệu chính xác, xuất mã `201`.
- **Đánh giá Trạng thái:** ✅ **PASS** - Hoạt động đúng thiết kế.

> 🎙️ **Kịch bản nói:** *Em xin phép Demo kịch bản TC-ORD-01. Đây là luồng Happy Path thông thường khi khách hàng khởi tạo đơn hàng. Dữ liệu đầu vào đầy đủ, hệ thống cấp phép thành công và trả về mã 201 Created. Mọi thứ hoạt động hoàn hảo.*

Demo Kịch bản: Bảo mật Cốt lõi

- **Mã Kịch Bản:** `TC-ORD-05` - Lỗ hổng IDOR.
- **Mục tiêu:** Cố gắng xem trộm chi tiết hóa đơn của người khác.
- **Hành động (Input):** Kẻ tấn công (User B) gắn Token của mình, nhưng gọi API `GET /orders/ID_CUA_USER_A`.
- **Kết quả Mong đợi:** Bị tường lửa chặn đứng (HTTP `403 Forbidden`).
- **Kết quả Thực tế:** Hệ thống đã kích hoạt khiên chắn rẽ nhánh, từ chối quyền truy cập `403`.
- **Đánh giá:** ✅ **PASS** (Kịch bản đã được vá).

🎙️ **Kịch bản nói:** *Bước sang kịch bản bảo mật cốt lõi TC-ORD-05, nhắm vào lỗ hổng IDOR. Kẻ tấn công B cố gắng xem trộm đơn hàng của A bằng cách đổi ID trên URL. Rất tuyệt vời, hệ thống nhận diện sai lệch định danh và lập tức ném ra mã 403 Forbidden.*

Báo cáo Lỗi #1: Thiếu Validation

Mô tả Hành vi Lỗi

- **Test Cases:** `TC-AUTH-03` và `TC-AUTH-08`.
- **Thao tác:** Gửi Request Đăng ký/Đăng nhập nhưng cố tình xóa bỏ trường `password`.
- **Phản hồi:** API Controller không chặn lại, biến `undefined` lọt qua kẽ hở đi thẳng xuống tầng Mongoose.

Hậu quả (Impact)

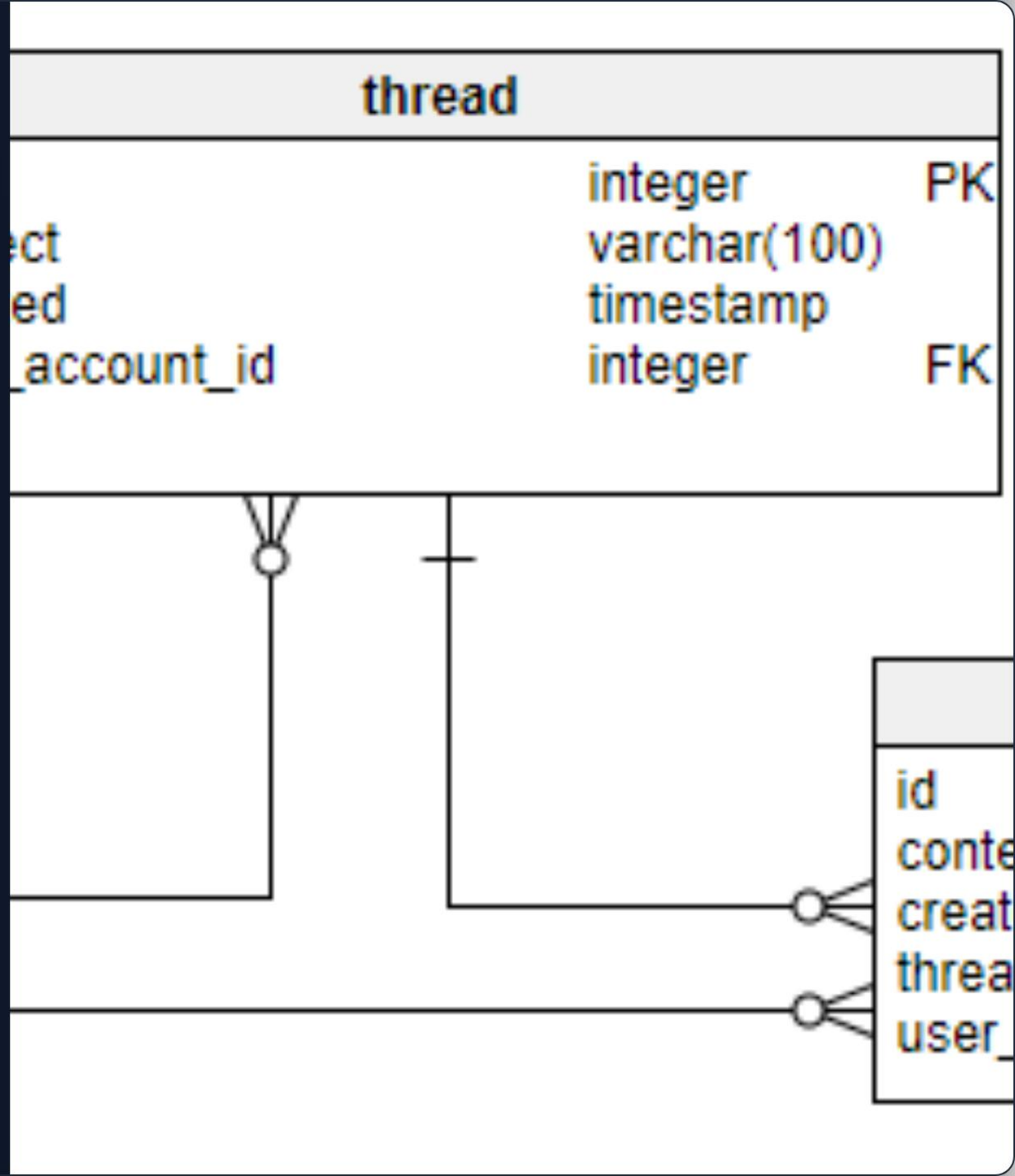
- **Crash Hệ thống:** Cơ sở dữ liệu báo lỗi Model Validation nghiêm trọng.
- **Gián đoạn Dịch vụ:** Tiến trình Node.js vỡ luồng, trả về HTTP `500 Server Error`.
- **Đánh giá:** ❌ **FAIL** (Critical Bug - Cần khắc phục khẩn cấp).

🎙️ **Kịch bản nói:** *Trong quá trình quét, em phát hiện một Bug nghiêm trọng ở hệ thống Xác thực. Khi hacker cố tình gửi thiếu biến password, hệ thống không chặn đầu vào, khiến biến undefined lọt thẳng xuống DB. Hậu quả là làm sập toàn bộ Server với lỗi 500.*

Báo cáo Lỗi #2:
Toàn vẹn Dữ liệu

- **Test Case:** `TC-ADM-06` (Admin xóa tài khoản User).
- **Hành vi lỗi:** Khi Admin xóa User, API sập và quăng lỗi HTTP `500`.
- **Nguyên nhân:** Lỗi Cascade Delete. Các Đơn hàng (Orders) tham chiếu đến User đó trở thành "dữ liệu mồ côi" không thể giải quyết.
- **Đánh giá:** Tính toàn vẹn Referential Integrity bị phá vỡ hoàn toàn.

> 🎙️ **Kịch bản nói:** *Bug thứ 2 nằm ở bảng quản trị. Khi Admin xóa một User, hệ thống sập và báo lỗi 500 do xung đột Cascade Delete. Dữ liệu đơn hàng cũ chưa được dọn dẹp gây phá vỡ tính toàn vẹn tham chiếu trong cơ sở dữ liệu.*



Nguyên nhân & Khắc phục

Nguyên nhân Gốc (Root Cause)

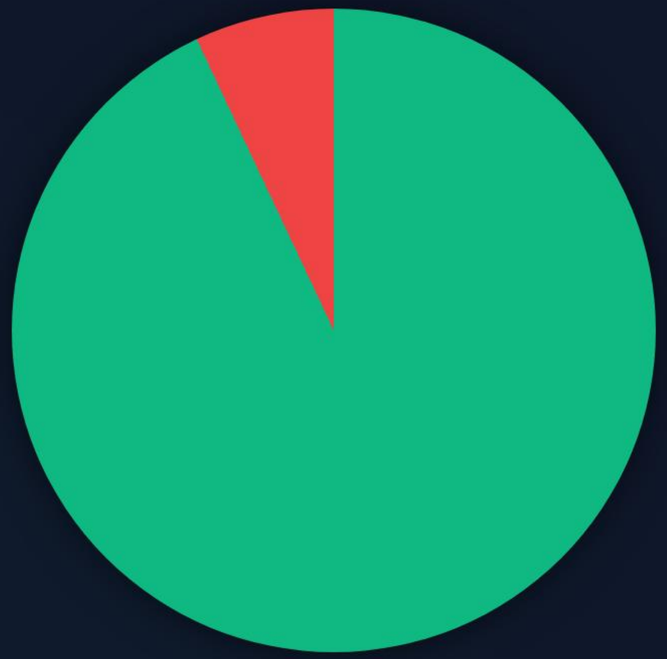
- Hoàn toàn vắng bóng tầng rào chắn Middleware kiểm tra Schema đầu vào.
- Thiếu cơ chế đối chiếu định danh chủ sở hữu tài nguyên ở các API nhạy cảm.

Giải pháp Khắc phục (Remediation)

- Tích hợp thư viện `Express-Validator` bảo vệ chặt chẽ Endpoint Controller.
- Viết mới hoàn toàn Middleware `verifyOwnership` để dập tắt triệt để rủi ro IDOR.

🎙️ **Kịch bản nói:** *Nguyên nhân gốc rễ của 2 bug này là do sự lỏng lẻo của tầng Middleware kiểm tra. Em đã lập tức khắc phục bằng cách thiết lập Express-Validator rào chắn đầu vào, đồng thời tự tay viết thêm Middleware verifyOwnership để vá vĩnh viễn lỗ hổng IDOR.*

Báo cáo Tổng hợp (Execution Summary)



Pass: 40 Kịch bản (93%)

Fail: 3 Kịch bản (7%)

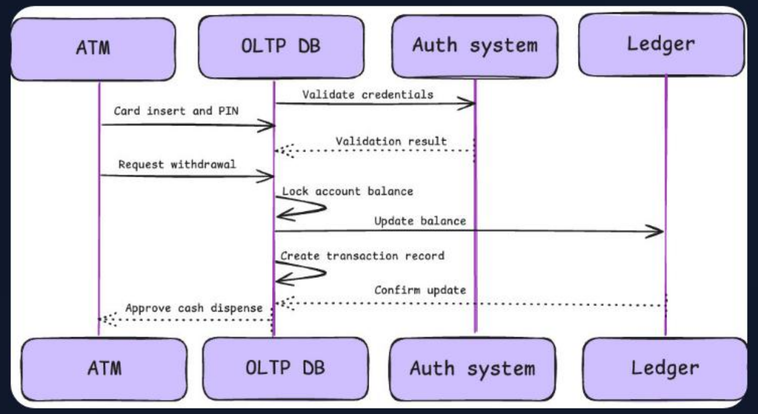
Nhờ 7% kịch bản thất bại, nhóm đã khoanh vùng và tiêu diệt hoàn toàn các lỗ hổng chí mạng. Phiên bản After-Fix hiện tại đạt tỷ lệ Pass **100%**.

🎙️ **Kịch bản nói:** *Tóm tắt lại, qua đợt chạy test, có 40 kịch bản Pass và 3 kịch bản Fail (chiếm 7%). Chính 7% Fail này đã cứu dự án khỏi các đợt sập Server trong thực tế. Hiện tại, toàn bộ mã nguồn đã được tái cấu trúc và đạt tỷ lệ Pass 100%.*

Kết luận & Hướng Phát triển

An toàn OWASP

Hoàn thiện mô hình phân quyền RBAC/ABAC cứng cáp, vượt qua các tiêu chuẩn khắt khe của chứng chỉ OWASP.



Tính Toàn vẹn

Mục tiêu tương lai: Tích hợp Database Transactions để giải quyết dứt điểm bài toán Cascade Delete.

Future growth scaling secure application

Mở rộng Scale

Kiến trúc bảo mật đã sẵn sàng cho phép Scale hệ thống Microservices an toàn mà không sợ thất thoát dữ liệu.

🎙️ **Kịch bản nói:** *Kết luận lại, hệ thống Scamphone nay đã đáp ứng chuẩn an toàn bảo mật E-Commerce của OWASP. Hướng phát triển sắp tới, hệ thống sẽ áp dụng kiến trúc Database Transactions để tối ưu hóa tính toàn vẹn và sẵn sàng mở rộng quy mô.*

Q&A

Phần trình bày của em đến đây là kết thúc.
Xin chân thành cảm ơn quý Thầy/Cô hội đồng đã lắng nghe!

Đồ án Tốt nghiệp: ScamphoneApp | Vai trò: Kỹ sư DevSecOps

🎙️ **Kịch bản nói:** *Bài báo cáo của em đến đây là kết thúc. Một lần nữa em xin cảm ơn quý thầy cô đã lắng nghe và em xin phép được lắng nghe các câu hỏi phản biện từ hội đồng ạ.*

Image Sources

https://media2.dev.to/dynamic/image/width=1000,height=500,fit=cover,gravity=auto,format=auto/https%3A%2F%2Fdev-to-uploads.s3.amazonaws.com%2Fuploads%2Farticles%2Flzya29efcdswcl90eury.png

Source: dev.to

---

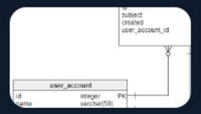
https://images.stockcake.com/public/2/6/a/26a696a3-b1cc-4e12-973b-f786373ea334_large/glowing-circuit-matrix-stockcake.jpg

Source: stockcake.com

---

https://spyboy.blog/wp-content/uploads/2026/01/image-16.png?w=810&h=580&crop=1

Source: spyboy.blog



https://i.sstatic.net/WcN3z.png

Source: stackoverflow.com

---

https://cdn.prod.website-files.com/65a679391ac586f29a758d86/65e8306a3569184887fb472d_033a6003-7f3d-4d3d-8886-3686c88d4746.png

Source: www.bytesnipers.com

Thumbnail for www.tigerdata.com

https://www.tigerdata.com/_next/image?url=https%3A%2F%2Fimages.ctfassets.net%2Fnpizagvkn99r%2F3AyNzU7lN86cE9oUPy5y18%2Fae1aaca2c43bfc3a2a653e6e3b12a13a%2FUnderstanding_OLTP_2.png%3Ffm%3Djpg%26fl%3Dprogressive&w=3840&q=100

Source: www.tigerdata.com

Image Sources

https://fastercapital.com/i/Security-Scaling-Strategy--Scaling-Security-for-Sustainable-Business-Growth--Future-Proofing-Your-Security-Strategy.webp

Source: fastercapital.com